Agentic Engineering: Building
and how to choose the model.
design is the answer to two
is EMERGENT CONTROL: the model
The route is fixed in code; the
other: summarise ten documents
(graph.py) graph.add_edge("draft",
model Use autonomy when you do
wrong, everything downstream is
lost, and decisions conflict. Many
has always used for parallel work, a
merges back to the main line. Conflicts
memory, do not hope for it. - Durable state
start of each run the agent fetches the task,
Persist the facts that must survive;
is a replaceable part. Choose it to the task, not
routine steps. Mixed-model pipelines
arguing about which is best. - Measure the
conditional). - No: emergent control. Use
agent. You end up with a mostly-explicit
a report with the same steps every time): a
request, send a ticket to the right handler,
repeats until done, bounded by a turn limit.
guard against going in circles. - Several
judge, a second model that reviews it; and check the judge.
Evaluation: is it working? You cannot improve what you
right answer: check it with code (exact match, a schema, a
the simplest version alongside the complex one. In one
cannot justify with a measured gain. ## 9. Optimisation: where
do. 2. Tools. What each agent can actually do. The right tool, well
the results back. - Run the loop: repeat gather-and-act until the
calls. A healthy loop is: gather context, take action, re-run the loop,
Intake: one message (from a phone, Slack, or a client) creates a card
structure, and writing. Then a judge, a second agent seeing the work for
running a different model (for example Opus,
status, so you can see what each agent did and
is. 4. If not, draw the workflow (sequential,
memory: keep durable state outside the agents
the output is subjective. 8. Add a judge for

Multi-Agent Systems A
The claim throughout: these
questions: 1. Who controls the flow?
decides at run time. Most real systems live in
model only fills in each step. Three shapes cover
at once, then combine. ### Conditional Read the
"check") # sequential: draft, then check
NOT know the steps, when the path only shows up by
wrong. ### Handoff Each agent decides who gets the work
voices, one thread, no owner. Autonomy buys flexibility and
copy each plus a system around it: - A copy each.
are settled at merge time, and the work comes
lives OUTSIDE the agent: a task board (NEW, PICKED
the relevant files, and prior results, and keeps only
recompute or re-fetch the
the task to the model. - Start
are normal. - When quality
choice: compare quality, tokens, and calls across models
autonomy. The answer shows up only by trying. And
pipeline and one or two ntelligent joints, which is
sequential workflow. Route t in code; each step is a model
choose a tool): a conditional workflow. One model reads the
- A goal that splits into sub-goals (build a feature, run a
candidates to choose among (design a page, draft
Most real applications combine these: a fixed pipeline
cannot measure. Build the evaluation before you scale the
passing test). Use a judge when it is a judgement
published test, a single model call beat a team of
most of the value is When the system works but is
described, beats a clever plan over the wrong tools.
work is done or a limit is hit. - Verify: before finishing, the
verify the work, with a turn limit so it cannot run away. ##
on a task board with three columns, NEW, PICKED UP,
the first time, reads the report. After a few fix cycles it is
Sonnet, and a GPT model) building the same
why. ## 12. Build procedure Do this, in order: 1.
parallel, conditional) and write the route in
(a board, files), and give each agent
judgement-call outputs,

practical, build-ready guide to orchestrating AI agents, written for a
systems are not magic. They are ordinary software built
Who decides what happens next. 2. How do agents
between, an explicit skeleton with intelligence at the
almost everything. ### Sequential One step after
request and pick the route. Use it when the path
graph.add_conditional_edges("triage", route) #
trying. Each pattern has a failure mode; know it before
next: desk, then billing, then tech. Flexible. Risk: the
costs control. Reach for it only when a fixed route
Give every agent its own workspace, a git worktree
back together. Shared mutable state is where multi-agent
UP, DONE), saved blocks,
what it needs. - A long
rest. - Each agent gets its
with the smallest, cheapest model that passes your
is subjective (a design, a draft), run several models
on your test jobs before you commit. ## 7. How to
the rule that matters most: even then, start with a
cheaper, more reliable, and easier to debug than
call or a tool. - Independent subtasks (summarise
input and picks the route. - Open-ended work with
campaign): plan-based. A lead plans and hands
copy, propose options): a parallel fan-out to several
with one autonomous
system. 1. Write the test
call: a second model
agents on every axis:
not good enough, optimise in this order. Most of the value is
10. The agent loop Inside a single agent is a loop.
agent checks its own work. - Turn-limit guard: cap the
11. Reference architecture: a working system One
DONE. - Listener: watches the board, picks up a new
handed back to the human. During testing, plant a fault
deliverable. A copy each, no shared state. - Human at the
Define good: write the test jobs, including the awkward
code. 5. Find the one step that cannot be pre-scripted.
its own working copy; add claims and merges for shared
and grade the judge. 9. Optimise instructions first, then

machine to read and act on. It is general: it names the
with AI components. Design them the way
share state? How they share what they
joints. Name where each part of your system
another. Use it when the steps are known and
depends on the input: a triage step that sends a
conditional: triage picks the route ``` If you can
you choose it. ### Plan-based A lead agent plans the
work can go in circles, agents passing it back and
cannot express the task. ## 4. State: give each agent
(wt-a, wt-b). They never write over each other. -
systems break. Isolate it, claim it, merge it. ## 5.
files, a database. The agent reads from it and writes to it;
shared conversation is NOT memory. Leaning on
own working copy of mutable state (section 4). Agents
evaluation. Many tasks need only one call to a capable
on the same task in isolation, a sandbox each, and let a
choose One question decides the shape: do you know the
workflow. Lay down the steps you do know, run it, and
autonomy everywhere. ### By use case Match the pattern
many files, grade many answers, enrich many records): a
an unknown path (research a question, fix a bug, investigate
out, then checks and retries; guard its single point of failure. -
models or prompts, then a human or a judge picks the best. -
step, or a router that dispatches to several workflows.
jobs first. Decide what good looks like, and include the
reviews the output. And check the judge too, grade the
quality 9.7 against 9.3, tokens 355 against 15,343,
here, not in more agents. 1. Instructions. The
Understand it, and bound it. - Gather context: read
iterations, and raise a clear "max turns exceeded"
concrete system that puts the patterns together. It is
card, and starts the flow. Everything after is
on purpose (a quote changed to claim more than it
end: the person picks the best of the several
cases. 2. Start with one call to the smallest
Make only that step an autonomous agent, and
artifacts. 7. Choose the model per step:
tools. 10. Keep a human at the end for the pick or

patterns for building agents and, for each kind of application, shows which
you design software, and add intelligence only where it earns its
know. Each answer sits on a spectrum. At one end is EXPLICIT
sits before you build it. ## 2. Workflow patterns: explicit
ordered: draft, then check, then publish. ### Parallel Fan out,
ticket to the right handler. Frameworks such as LangGraph
draw the graph, write it in code. The model is a node, not the
work, hands out tasks, checks the results, and retries failures,
forth with no one finishing. ### Group chat All agents share one
ts own copy The clash: two agents both write the same file,
Claims. Each unit of work is claimed by exactly one agent (Task
Memory and context An agent does not remember on its own
if the agent restarts, the
an ever-growing chat is the
share through claimed,
model. - Escalate only
human or a judge pick the
steps? - Yes: explicit
watch where it breaks.
to the work: - Known, fixed
parallel workflow. Fan out,
a system): an autonomous
A chain of specialists
A deliverable that must be
Start from the case closest
awkward cases, the inputs
grader against a few
model calls 1 against 3 or
prompt and the guidelines
the task, fetch what you
rather than loop forever. -
an example of the general
automatic. - Report block:
says) and confirm the judge
outputs. That is the only
model that could work.
pick its pattern (plan-based,
cheapest that passes; a
the approval, and treat the

pattern to reach for, how to hold
place. ## 1. The two
CONTROL: you decide
control, routed in code Use
then bring the results
express exactly this. A
orchestrator. ## 3.
like a project manager.
conversation. Easy to wire
guide.md. One overwrites
1 to A, Task 2 to B). No two
between runs; it gathers
state is still there. -
group-chat failure mode:
merged artifacts, not a
where it breaks: a stronger
best. Different models have
control. Use a workflow
Replace only the broken
pipeline (document
bring the results back. -
agent loop. The agent
(intake, then billing, then
correct (a legal brief, a
to yours, build it explicitly,
that are easy to get wrong.
known-good and
more. For a simple task,
each agent follows.
need. - Take action: the
Plan between calls: for
shape, not the only one.
an agent gathers sources,
catches it. - Fan-out: split
human step. - Everything is
Measure quality, tokens,
handoff, or group chat)
stronger one only for the
whole thing as software: log

state and memory,
questions Every multi-agent
everything up front, in code. At the other
a workflow when you already know the steps.
back. Use it for parts that do not depend on each
workflow is just a graph of steps: ``` # LangGraph
Autonomous patterns: emergent control, routed by the
Risk: the lead is a single point of failure. If its plan is
up. Risk: as the conversation grows, cost climbs, details are
the other, and work is lost. The fix is the one software
agents take the same task. - Merges. Each workspace
the context it needs each time. Design the
Working context is gathered per turn: at the
cost climbs, details are lost, decisions conflict.
shared scratchpad. ## 6. Model choice The model
model for the one hard step, a cheaper one for the
different strengths; trying a few and choosing beats
(sequential, parallel,
step with an autonomous
processing, data extraction,
Routing or triage (classify a
gathers, acts, verifies, and
technical): handoff. Each agent picks who is next;
published report, anything cited): a worker plus a
and add autonomy only where it breaks. ## 8.
2. Grade two ways. Use code when there is a
known-bad examples, or it will quietly drift. 3. Run
one model call often wins. Do not add agents you
Sharper instructions fix more than extra agents
model requests tool calls; you execute them and feed
hard tasks, let the model plan its next step between tool
One message runs it all, and a human only at the end. -
does the work, and runs its own checks for accuracy,
the work into several isolated sandboxes, each
logged: a live event stream and an explicit per-step
and calls. 3. If one call is enough, ship it. It often
with its failure-mode guard. 6. Design the
hard step; several in parallel when
every step, cap every

loo han eve failu ## 13. Pro pts you can reus Wo
age "Yo are {ro Goa {tas Tools ava able {to . Ga he con you need the
do the wor Bef you finis ver you out t aga the che {accu , st
writ If you can fini within {N} steps, sto and report exactly what is blockin you. Do not guess. Judge agent: "You are a reviewer see g this work for the first time. The task was {tas }. The sourc is: {sourc Check the out for
accuracy (every quo and nu must appear in the sou e, unchange structure ({required section), and wri ing ({ru es}). List each problem and a concrete fix. Do not rewrite he work; only judge it." Rou er (a conditional ste): "Read the req below. Choose exa y on rou e from this set: {route }. Return only the rou nam
nothing els Req est {requ st}. Lea or pla (pla ased): "Br ak his goal into independ t tasks tha can be worked in par : {goal}. As gn eac tas to one worker. Afte each wo ker retu ns, check its resu ret a failed task up to {N} tim s; the assemble the fina resu Report wh tas fail
14. Gu elines to ke - Sta exp add em rge on w ere the wo breaks. - Pre er one model call; ust every add d agent with a mea ured win. - Choose the mo el to the tas cheape that pas escalate only the har ste , try several when qua y is sub - Keep dur
stat out de the agent ; give eac its ow cop set e con icts at me A long ch t is not me ory. - Use a judg for judg men call and che the judge. - Defi e good with tes obs bef re you bui - Mos gai com from ins ctio s and too s, not mor agents. - Keep a hum
at the end. - It is soft name the control and the tate log it, cap the loops, handle failu . ## Pro nance Th page is an orig writ synthesis by typog aphics-ar org — the field's common agent-bu ding patt set out for a machine to read and act on. It is set as a Typ hic
Art one ger: eac sec carri its own hash, so you can verify a single quoted sec agai t the sealed documen and the who e page carries the issuer' Ed25519 sign ture